Reporte Técnico de Actividades Práctico-Experimentales Nro. 005

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Carlos Granda Juan Veintimilla Fabrico Ruiz Maria Chuico
Asignatura	Sistemas Distribuidos
Ciclo	6 A
Unidad	1
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	5
Título de la Práctica	Mensajería Distribuida con RabbitMQ y MQTT
Nombre del Docente	José O. Guamán Q.
Fecha	20 de octubre de 2026 23 de octubre de 2026
Horario	Martes 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

Implementar la capa IoT de un sistema de monitoreo de flota logística utilizando el protocolo MQTT: configurar un broker Mosquitto en Docker, simular sensores que publican telemetría en topics jerárquicos con paho-mqtt, implementar un suscriptor que almacene los datos en SQLite y genere alertas por umbral, y construir un servicio puente que publique las alertas en colas de RabbitMQ.

- Comprender los fundamentos del protocolo MQTT y su aplicación en entornos IoT
- Configurar e instalar un broker Mosquitto para mensajería publicador/suscriptor
- Implementar publicadores y suscriptores MQTT en Python usando paho-mqtt
- Diseñar topic hierarchies para organizar la telemetría de una flota vehicular

- Entender el protocolo AMQP 0-9-1 y el modelo de intercambio de RabbitMQ
- Configurar exchanges, colas y bindings en RabbitMQ

3. Materiales y Reactivos

Guia practica

Software

- Python
- paho-mqtt
- pika
- Java JDK 17
- Spring Boot
- Docker
- Docker Compose
- Mosquito
- RabbitMQ
- SQLite3
- IDE
- Postman
- Navegador web

Hardware

- Computador

4. Procedimiento / Metodología Ejecutada

Describa brevemente los pasos que siguió durante la ejecución de la práctica.

5. Resultados

Broker MQTT funcional: Un broker Mosquitto corriendo en Docker, capaz de recibir y distribuir mensajes de múltiples publicadores a múltiples suscriptores.

Simulador de sensores IoT: Un script Python funcional que simula datos de telemetría de tres vehículos (GPS, temperatura, combustible) y los publica periódicamente en topics MQTT jerarquicos.



Figura 1. Prueba de publicación y suscripción mediante MQTT

En esta captura se puede observar el funcionamiento del protocolo MQTT utilizando el broker Mosquitto. En una terminal se ejecuta el comando `mosquitto_sub`, que permite suscribirse a un topic y esperar los mensajes publicados. En otra terminal se utiliza `mosquitto_pub` para enviar mensajes al topic correspondiente. Como resultado, se observa que el suscriptor recibe el mensaje enviado por el publicador. Esto demuestra que el broker recibe las publicaciones y las distribuye a los clientes que están suscritos al topic indicado.

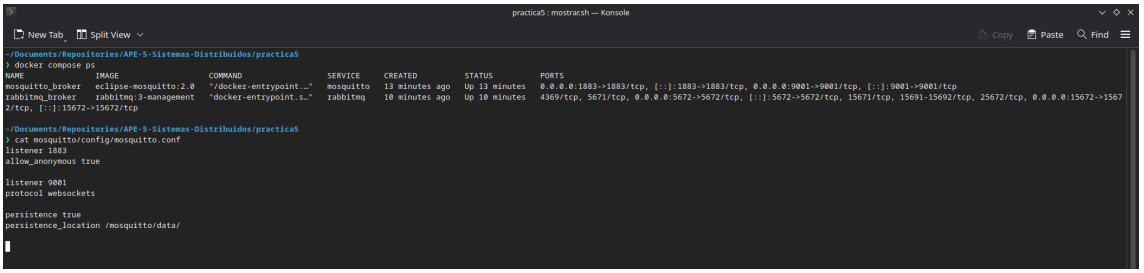


Figura 2. Comunicación de un publicador con múltiples suscriptores

En esta captura se demuestra la capacidad de MQTT para distribuir un mismo mensaje a varios suscriptores. Para realizar la prueba, se ejecutan dos procesos `mosquitto_sub` en terminales diferentes, ambos suscritos al mismo topic. Posteriormente, desde otra terminal se utiliza `mosquitto_pub` para publicar un mensaje en dicho topic. Como se puede observar, los dos suscriptores reciben el mensaje publicado. Esto permite comprobar que el broker Mosquitto puede distribuir una publicación a múltiples clientes suscritos, facilitando la comunicación entre diferentes componentes de un sistema distribuido.

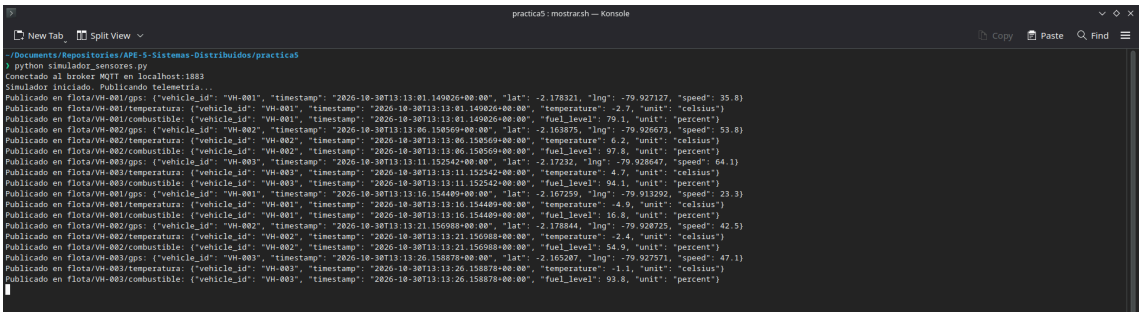


Figura 3. Simulación de sensores IoT mediante MQTT

En esta captura se puede observar la ejecución del script `simulador_sensores.py`, el cual simula el funcionamiento de tres vehículos que envían información a través del protocolo MQTT. Para ello, el simulador se conecta al broker Mosquitto y publica diferentes tipos de telemetría, correspondientes a la ubicación GPS, la temperatura y el nivel de combustible de cada vehículo.

Los datos se envían en formato JSON, lo que permite organizar la información mediante campos y valores para facilitar su lectura y procesamiento. Así mismo, las publicaciones se realizan repetidamente en distintos momentos, simulando la actualización constante de los datos de los vehículos. De esta manera, se comprueba que el sistema puede generar y transmitir información de varios vehículos mediante el broker MQTT, que posteriormente puede ser recibida y procesada por otros componentes del sistema.

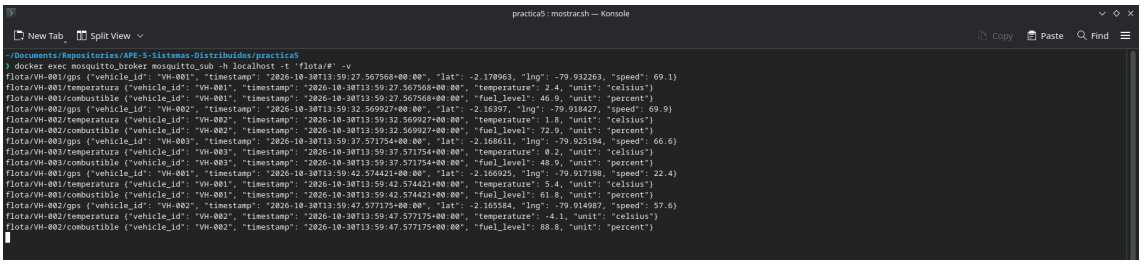


Figura 4. Topics MQTT de los tres vehículos.

La captura demuestra que el broker MQTT recibe y distribuye mensajes de los vehículos `VH-001`, `VH-002` y `VH-003` mediante topics jerárquicos para GPS, temperatura y combustible.

Suscriptor con almacenamiento: Un suscriptor Python que recibe datos de telemetría, detecta condiciones de alerta y almacena los datos en una base de datos SQLite con tablas separadas para cada tipo de sensor.

Servicio puente operativo: Un servicio Python que consume datos de MQTT y los publica en colas

RabbitMQ, filtrando unicamente los mensajes que requieren procesamiento (alertas).



Figura 4. Recepción de telemetría y generación de alertas mediante MQTT

En esta captura se observa la ejecución del suscriptor desarrollado en Python, el cual se conecta al broker Mosquitto para recibir los mensajes publicados por el simulador de sensores IoT. El suscriptor recibe la información de los vehículos mediante los topics correspondientes a los datos GPS, temperatura y combustible. Posteriormente, analiza los valores recibidos para identificar situaciones que requieren atención. Se genera una alerta cuando la temperatura supera los 4 °C o cuando el nivel de combustible es inferior al 20 %. De esta manera, se demuestra que el sistema no solamente recibe mensajes mediante MQTT, sino que también permite procesar la telemetría y detectar posibles situaciones de riesgo a partir de las condiciones establecidas.



Figura 5. Recepción de telemetría y generación de alertas mediante MQTT

En esta captura se observa el funcionamiento del suscriptor desarrollado en Python, el cual se conecta al broker Mosquitto para recibir los mensajes enviados por el simulador de sensores IoT. El suscriptor recibe la información de los tres vehículos mediante los topics correspondientes a GPS, temperatura y combustible. Posteriormente, analiza los valores recibidos para identificar condiciones que requieren atención. Se genera una alerta cuando la temperatura supera los 4 °C o cuando el nivel de combustible es inferior al 20 %. De esta manera, se demuestra que el sistema no solamente recibe mensajes mediante MQTT, sino que también procesa la telemetría y detecta posibles situaciones de riesgo según las condiciones establecidas.

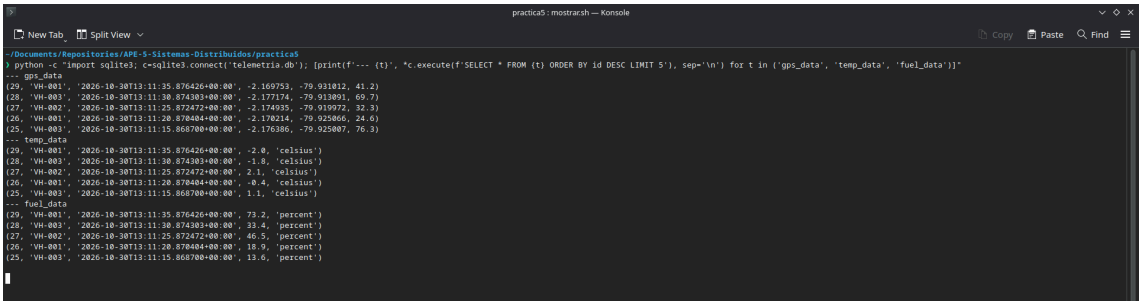


Figura 5. Consulta y almacenamiento de la telemetría en SQLite

En esta captura se presentan las consultas realizadas a la base de datos SQLite para comprobar el almacenamiento de la información recibida por el suscriptor. Los datos se organizan en tres tablas independientes: `gps_data`, que almacena la información relacionada con la ubicación de los vehículos; `temp_data`, que registra los valores de temperatura; y `fuel_data`, que guarda los niveles de combustible. Esta separación permite consultar cada tipo de telemetría de forma organizada y comprobar que la información se conserva en la base de datos para su posterior revisión y análisis.

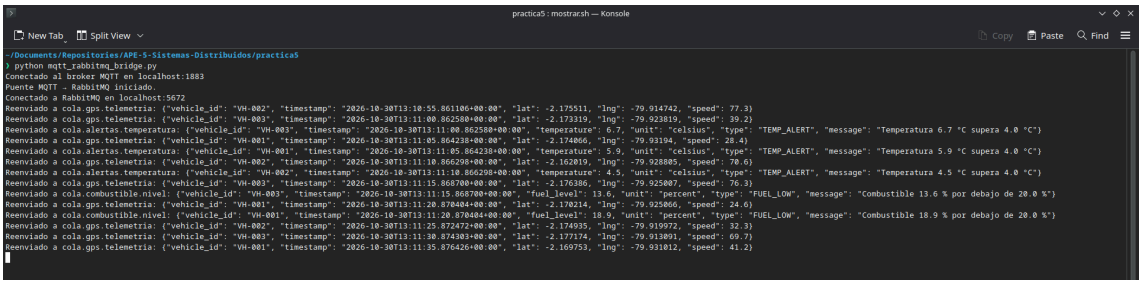


Figura 6. Comunicación entre MQTT y RabbitMQ mediante un servicio puente

En esta captura se observa el funcionamiento del servicio puente desarrollado en Python, encargado de comunicar el broker MQTT con el sistema de mensajería RabbitMQ. El puente se suscribe a los topics de telemetría de los vehículos, recibe los mensajes publicados y evalúa si corresponden a una alerta de temperatura o de combustible. Cuando se cumple alguna de las condiciones establecidas, el evento se publica en la cola de RabbitMQ correspondiente. En cambio, los mensajes normales de telemetría que no generan alertas se descartan para este proceso de reenvío. De esta manera, se demuestra cómo el servicio puente permite integrar ambos sistemas de mensajería y transmitir selectivamente los eventos importantes.

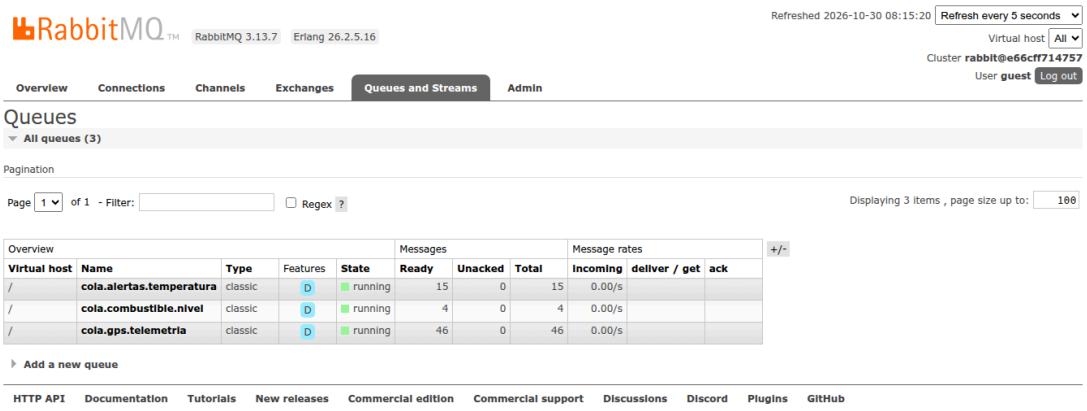


Figura 7. Verificación de las colas de alertas en RabbitMQ

En esta captura se observa la interfaz de administración web de RabbitMQ, desde la cual se puede comprobar el estado del servicio y la existencia de las colas utilizadas para recibir los eventos de alerta. También se pueden revisar sus nombres y las estadísticas de mensajes publicados, pendientes o disponibles, según la información que presente la interfaz. Esta evidencia permite verificar que RabbitMQ está funcionando y que se dispone de colas destinadas a recibir los eventos reenviados por el puente MQTT. Asimismo, complementa la captura anterior, ya que permite observar el destino de los mensajes de alerta dentro del sistema de mensajería.

6. Preguntas de Control

- **Explique las diferencias clave entre MQTT y AMQP en términos de protocolo, patrones de comunicación y casos de uso típicos. Por que MQTT es mas adecuado para IoT?**

  Ambos son protocolos se usan para el intercambio de mensajes entre dispositivos, aplicaciones y sistemas distribuidos ambos permiten comunicación independiente pero se diferencian en su arquitectura y en otros aspectos.

  En resumen se puede decir que MQTT es más adecuado para IoT porque fue diseñado para transmitir pequeños mensajes con una sobrecarga reducida, consumir pocos recursos y funcionar en redes con conectividad limitada. Su modelo Publisher/Subscriber facilita la comunicación entre numerosos dispositivos sin necesidad de conexiones directas entre ellos. Por otro parte, AMQP es apropiado cuando las aplicaciones necesitan mecanismos más avanzados de enrutamiento, gestión de colas y procesamiento confiable de mensajes.

  Es importante tener en cuenta que MQTT también ofrece seguridad de entrega mediante QoS, por lo que no significa que sea un protocolo poco confiable porque para determinar cuál es el más adecuado depende de los requisitos.

- **Por qué MQTT utiliza topic hierarchies (jerarquía de topics) mientras que RabbitMQ utiliza exchanges, colas y bindings? ¿Cuál es la ventaja de cada enfoque?**

La diferencia está en la forma en que cada protocolo organiza y distribuye los mensajes.

MQTT utiliza topics (temas) porque busca que la comunicación entre dispositivos sea sencilla y ligera. Los topics se organizan por niveles, como si fueran carpetas.

RabbitMQ utiliza exchanges, colas y bindings porque necesita organizar mejor los mensajes y decidir a dónde debe enviarse cada uno.

Las ventajas son que:

- **MQTT:** es más sencillo, consume menos recursos y facilita la comunicación entre muchos dispositivos. Es muy importante para cuando los sensores envían información constantemente.
- **RabbitMQ:** permite organizar mejor los mensajes, distribuirlos entre diferentes servicios y controlar cómo se procesan. Es muy importante para sistemas empresariales donde se necesita una gestión más avanzada.

En pocas palabras, MQTT facilita la comunicación entre dispositivos mientras que RabbitMQ ofrece mayor control sobre la distribución y el procesamiento de los mensajes.

- **Describa los tres niveles de QoS en MQTT (0, 1, 2). Que ventajas y desventajas tiene cada uno? En qué escenario del monitoreo de flota utilizaria cada nivel?**

  - **QoS 0:** el mensaje se envía una sola vez sin confirmación. Es rápido pero puede perderse, por lo que se lo usaría para enviar ubicaciones GPS.
  - **QoS 1:** garantiza que el mensaje llegue al menos una vez aunque puede repetirse, se lo usaría para alertas de velocidad o temperatura.
  - **QoS 2:** garantiza la entrega del mensaje exactamente una vez a nivel del protocolo MQTT. Es más lento pero evita duplicados por retransmisiones. Lo usaría para notificaciones de accidentes o emergencias.

  Por lo que se puede decir que MQTT es ideal para conectar dispositivos IoT porque es ligero y eficiente. Sus niveles QoS permiten elegir la seguridad de entrega según la importancia de los datos.

7. Conclusiones

- Fundamentos de MQTT y configuración del broker: se comprobó que el modelo publicador/suscriptor de MQTT desacopla a los emisores de los receptores. El broker Mosquitto, desplegado en Docker, distribuyó correctamente los mensajes de varios publicadores a varios suscriptores sin que estos se conocieran entre sí.

- Publicadores, suscriptores y jerarquía de topics: los scripts en Python con paho-mqtt simularon y recibieron la telemetría de tres vehículos. La jerarquía flota/{vehiculo_id}/{tipo_sensor}, junto con los comodines + y #, permitió filtrar la información por vehículo o por tipo de sensor de forma sencilla.
- Almacenamiento y alertas: el suscriptor guardó la telemetría en SQLite, en tablas separadas por sensor, y detectó en tiempo real las condiciones de riesgo: temperatura mayor a 4 °C y combustible menor al 20 %.
- AMQP y RabbitMQ: el servicio puente reenvió el GPS y las alertas a colas durables con mensajes persistentes. En la interfaz web se verificó que los mensajes se acumulaban hasta ser consumidos, lo que demuestra la entrega confiable de AMQP.
- Integración de ambos protocolos: MQTT resultó adecuado para la capa IoT por su ligereza, y RabbitMQ, para el procesamiento interno por su enrutamiento y persistencia. Combinarlos mediante un puente da una arquitectura distribuida y escalable, lista para incorporar los microservicios de la Parte 2.

Declaración de Uso de Inteligencia Artificial

Se utilizó la herramienta de Inteligencia Artificial Generativa de Claude como apoyo de tutoría técnica para la configuración del entorno en Linux Ubuntu (resolución de conflictos de puertos de Docker, creación de entornos virtuales venv), la implementación de scripts en Python (suscriptor_sqlite.py y mqtt_rabbitmq_bridge.py) y la estructuración de las respuestas teóricas del informe. Todo el código generado fue revisado, probado y ejecutado directamente en mi equipo de trabajo."

Prompt principal utilizado:

"Actúa como un tutor de Sistemas Distribuidos. Guíame paso a paso para configurar el entorno en Ubuntu, resolver errores de terminal (Docker, Python venv) y desarrollar los scripts de la práctica APE 005 para registrar telemetría vehicular en SQLite y redirigir alertas hacia RabbitMQ desde un broker MQTT Mosquitto."